Guarded proof cores and smaller certificate interfaces

Review of three new K215 packets and further simplifications

Research contribution for Vladimir Reshetnikov's ProveIt project

21 October 2026 UTC

Conclusion. Two further reductions are worth formalizing: a 189-vertex guarded arithmetic proof core, obtained from the new 367-vertex affine core, and a source checker whose only lookup requirement is coverage by checked entries. The latter supports a size-free positive trie with 16.37% fewer measured lookup-node visits than the incoming balanced layout. Neither result is a machine-state reduction or a measured build speedup.

Exact scope. Review cutoff `aa66d0a33f1f` has parent `0359ebda60aa` and adds exactly three incoming ZIPs, with no production-source change. The preceding seven-page follow-up remains unchanged. Production phase five still uses the ordinary 2,240-entry Lean certificate and the explicit 512-step decoder; do not substitute an older vendored checker. This report reviews the newly supplied evidence and develops additional ordinary proofs, rather than recounting earlier simplifications.

1 The evidence now supports stronger starting points

Normalized Rocq is no longer merely a proposed port. The checked packet records twelve fresh Coq 8.18.0 source/dependency compilations and successful recursive `coqchk` on `NormalizedSource215`. Its 1,426-entry theorem is

$$\forall R, \quad \mathrm{Halts}(p215, ([], R)) \iff \mathrm{Halts}(dagPerm, ([], R)).$$

The same-name public facade and an explicitly typed client are separately recorded as compiled and closed under the global context. All source/log bindings and all 1,426 path/successor rows were independently checked. Production `Prog215.v` and `DagPerm.v` at the cutoff are byte-identical to the supplied inputs. The remaining task is production integration under Rocq 9.1 and full endpoint replay, not a second port. The recorded 80.632 compiler seconds and separate 521.235 recursive-check seconds are not a controlled speed comparison.

Decoder framing and composition now have recorded proofs. Three Lean 4.32.0 modules establish arbitrary-tape finite-window refinement, positive local blocks and generic `Chain` composition. The instance checks 281 blocks and 234 guarded roles. The missing theorem is construction of the `Chain` from the source/role structure, not arbitrary-tape soundness from scratch. Section 5 gives its induction and exact boundary.

Other supplied proofs remain distinct. The refinements packet records a balanced 2,240-entry Rocq source theorem, a shared native two-marker checker and concrete Cantor credit. The balanced recursive audit covers its generic checker, not its concrete source theorem. The checked packet's normalized source audit does cover the concrete source closure. Both credit developments discharge concrete syntactic codec/counting obligations, not the physical scheduler adapter.

Evidence discipline. No Lean, Coq or Rocq compiler was run in this review; no received executable was executed. Recorded builds have independently verified source/log bindings. New finite checks were executed with independently written Python. All new formal ideas in this report, including the guarded core, positive trie soundness extension and automatic `Chain`, remain *uncompiled*. There is no new 214-state machine.

2 Two small loop proofs reduce the affine core further

The incoming graph compiler produces identical 1,426-vertex, 2,850-edge canonical checkpoint graphs from both 4,751-leaf source trees. It retains consumed leaf paths before break/continue arithmetic and uses fixed-width, wrapping `incPath`; neither suffix deletion before the operation nor carry-extending increment is sound. Independent reconstruction binds the graphs to the current tree literals. The generic source-to-graph bridge must canonicalize decoded consumed paths, rather than identify arbitrary raw pointers with canonical leaf strings; it still needs formalization.

The first contraction. At a test of register r, suppose its positive self-loop decrements R_r and then adds a natural vector c, with $c_r = 0$. Suppose its zero exit adds d and leaves the node. Induction on R_r gives, after $R_r + 1$ checkpoint steps,

$$R'_r = d_r, \qquad R'_j = R_j + R_r c_j + d_j \quad (j \neq r).$$

This handles arbitrary initial registers, including a nonzero exit increment d_r. The packet contracts 1,059 such drains to a 367-vertex affine graph with 33 distinct homogeneous 10×10 matrices. Every positive affine branch means *decrement first, then apply the matrix*. Matrix order cannot be inferred from a graph-size comparison.

The new observation. Exactly 174 of those retained tests have the following complete positive self-loop action, already including that decrement:

$$R'_1 = 0, \qquad R'_6 = R_1 + R_6 - 1, \qquad R'_7 = R_7 + R_1 + R_6 \qquad (R_6 > 0).$$

Registers $0, 2, 3, 4, 5, 8$ are unchanged. Write $a = R_1$, $n = R_6$, $z = R_7$ and $T(k) = k(k+1)/2$.

Proposition 2.1 (Guarded triangular loop)**.** *Immediately before taking the zero exit, the loop produces the register file*

$$S = \begin{cases} R, & n = 0, \\ R[1 \mapsto 0,\ 6 \mapsto 0,\ 7 \mapsto z + T(a+n)], & n > 0. \end{cases}$$

The number of positive self-loop bodies is zero if $n = 0$ and exactly $a + n$ otherwise. Updates in the display are simultaneous.

Proof. The zero case executes no body. In the positive case put $s = a + n > 0$. The first body gives $(R_1, R_6, R_7) = (0, s-1, z+s)$. Thereafter each positive body decreases R_6 by one and adds its previous positive value to R_7. Induction using $T(k+1) = T(k) + (k+1)$ gives $s - 1$ further bodies and final value $z + s + T(s-1) = z + T(s)$. Every other coordinate is fixed by each body. The decreasing measure is $R_1 + R_6$, not R_6 alone: the latter can increase in the first body. Once a body has executed, $R_1 = 0$, so the loop cannot stop early with positive remaining sum. □

For node v, let Z_v be its literal zero-edge affine map and w_v its target. The complete summary is $(w_v, Z_v(S))$, not just S. It takes one outer affine-graph step if $n = 0$, and $a + n + 1$ if $n > 0$. These are *not* primitive register-machine or TM transition counts.

A necessary guard, not a cosmetic case split. At node 36, take $R_1 = 2$ and every other register zero. The true exit to 62 has $R'_6 = 2$, $R'_7 = 0$. Applying the positive formula when $R_6 = 0$ instead gives $R'_6 = R'_7 = 3$. No clean-scratch or global reachability hypothesis can be silently inserted into the advertised all-register contract.

3 Composition yields 189 proof vertices with positive progress

Four further nodes, 4567, 4585, 4668 and 4733, test register 6. Their positive self-loop decrements R_6, clears R_5 and leaves every other register unchanged.

Proposition 3.1 (Conditional clear)**.** *For initial $n = R_6$, the state before the zero exit is R when $n = 0$, and $R[5 \mapsto 0, 6 \mapsto 0]$ when $n > 0$. There are exactly n positive bodies, followed by the literal zero-edge map once.*

Proof. For $n = 0$ no body is taken. For $n > 0$, the first body clears register 5; the remaining $n - 1$ bodies leave it zero and count register 6 down to zero. Induction on n proves termination and all unchanged coordinates. Thus the complete macro takes $n + 1$ outer affine steps. Clearing register 5 at $n = 0$ is wrong; at all four literal zero exits this error remains observable. □

Delete these 178 nodes. A retained edge performs its original test, conditional decrement and affine map, then its checked finite list of total-loop summaries including their zero exits. The resulting circuit has

$$367 - 174 - 4 = 189 \text{ vertices} \quad = \quad 188 \text{ tests} + 1 \text{ halt}, \qquad 376 \text{ edges}.$$

The 182 loop references have maximum path length seven. Both empty-pointer entries decode directly to retained checkpoint 0 without updates. The unique halt, literal source ordinal 4736, remains retained with no successors. Failed lookup, malformed input or exhausted fuel must reject, never invent a halt.

Proposition 3.2 (Finite path contraction)**.** *Let the original transition be deterministic with no step from literal halt, and let retained configurations preserve and reflect halt. Suppose every nonhalt retained configuration has a contracted successor and every contracted step has a nonempty finite original realization with nonterminal interior and its exact final register file. Then contraction preserves halting and final registers from every retained configuration.*

Proof. Expand each finite contracted run by concatenating its exact original runs. Conversely, for a terminating original run at a nonterminal retained node, the certified expansion is its unique deterministic prefix. It cannot pass an earlier halt because its interior is nonterminal. Remove this prefix and repeat. Strictly positive prefix length decreases the remaining original run length, so induction reaches the retained halt. Exact boundary configurations preserve every final register in both directions. □

The retained test supplies positive progress even for an empty summary list. Each affine edge expands to a positive checkpoint run; its test consumes a source instruction. The semantic bridges must establish this. All tests and updates use registers 0 through 8; registers $j \geq 9$ are an arbitrary unchanged frame, not assumed zero.

Do not duplicate the termination certificate. Finite checked paths prove composition terminates; retained starts need no separate ranks. All 178 removed vertices occur among the 182 references, so existing path suffixes normalize every removed start too. First-level paths likewise cover all 1,059 drains. Another graph needs this coverage or additional checked entry paths for an all-start theorem.

Representation and acceptance. Keep guarded circuits, not expanded polynomials: these maps are not affine, and 189 is not a TM state count. Compile the loop inductions, source bridge and path theorem before adoption. Current canonical graphs already agree, making direct equality the simpler baseline. Reuse recursive `Cantor.tri`; `pair162` needs a different-boundary and higher-register frame adapter.

4 Make checked lookup coverage the source checker interface

Source bisimulation needs retrieved entries to satisfy their checked local conditions, not ordinal indexing, flattened order, balance or unique checkpoint values.

Fix programs P, Q, a lookup $G : K \to \mathrm{Option}(C)$ and a finite family of checkpoints. Let $\mathrm{Good}_G(c)$ be the *existing* local source condition for c, with all successor lookups performed by this same G. Preserve the exact segment/path semantics and the initial segment check. The needed interface is

$$G(k) = \mathrm{some}(c) \quad \Longrightarrow \quad \mathrm{Good}_G(c).$$

Checking a different family from the possible outputs of G can admit an unchecked decrement/halt mismatch.

Proposition 4.1 (Size-free positive trie coverage)**.** *Let a trie be empty or* $\mathrm{Node}(c, L, R)$*, with a checkpoint at* every *nonempty node. For positive binary keys define*

$$\begin{aligned} G_{\mathrm{Node}(c,L,R)}(\texttt{xH}) &= \mathrm{some}(c), \\ G_{\mathrm{Node}(c,L,R)}(\texttt{xO}\,p) &= G_L(p), \\ G_{\mathrm{Node}(c,L,R)}(\texttt{xI}\,p) &= G_R(p). \end{aligned}$$

Empty lookup fails. If a predicate F is checked at every payload, then every successful lookup returns a payload satisfying F, for every trie shape and key.

Proof. Induct on the trie. An empty trie cannot return a payload. At a node, key `xH` returns the checked root; keys `xO` p and `xI` p reduce to the respective subtree induction hypotheses. Apply the result with the *fixed whole-trie* predicate $F = \mathrm{Good}_{G_T}$, not a predicate that changes lookup when descending into a subtree. □

Repeat the inherited induction on a terminating source run. The initial check establishes the relation; coverage supplies the local check. Decrement consumes a source step, giving strict induction decrease; existing segment lemmas handle intervening instructions. Symmetric local checks yield reflection without another large evaluation. This requires a *new generalized theorem*, not deletion of `validb` from the compiled balanced theorem.

Same 2,240-entry certificate	Incoming balanced tree	New positive trie
Queried references	4,479	4,479
Lookup-node visits	54,514	45,591
Maximum successful visits	13	12
Stored cut or size fields	2,239	0
Successor binary digits	45,567	45,592
Stored cut binary digits	4,275	0

Reindex j as positive $j + 1$, initially key 1. All 4,479 semantic branches pass. Independent checking covers 16,383 keys through 14 bits and rejects seven corruptions. Savings are 8,923 visits (16.37%) and 4,250 counted binary digits, excluding constructors and sharing, not measured memory or elapsed time. Structural lookup consumes one key constructor per visited node, avoiding binary comparison/subtraction. Benchmark generation and elaboration too; the inferior leaf-only prototype is not recommended.

The coverage draft is uncompiled; generalized source soundness is unimplemented. Compile the concrete theorem, preserve the facade and rebuild both endpoints. The 1,426-entry lane additionally needs its canonical-path correspondence, not blind certificate substitution.

5 Derive decoder chains and connect concrete scheduler credit

What the decoder packet already proves. A local step fails on any unspecified read. A successful restored-window run therefore refines an arbitrary integer-indexed tape and restores its entire exterior as well as the observed cells. Each checked block has positive duration and advances the head by one. The recorded `chain_sound` composes such blocks, but assumes a `Chain` containing their membership and translated-window premises.

Proposition 5.1 (Construct the chain by child-node induction)**.** *Assume the literal structural check and all block checks succeed. At any checked role r, let the tape immediately preceding the head match its stored suffix, which has length one or two. Then the tape determines a finite route through internal decoder blocks to a primitive edge. Its internal prefix admits the required `Chain`; the primitive edge names the selected source leaf and its exact first transition.*

Proof. Induct on the role's source-node number. Read the current tape bit b and put $w = r.\mathrm{suffix} +\!\!+ [b]$. The structural check selects the corresponding source child and verifies its index is strictly smaller. If it is a leaf, the primitive edge checks both the leaf identifier and the literal machine action. No internal block is needed.

Otherwise the check supplies a target role s and checked block whose entry/exit states agree with r, s, whose word is a suffix of w, and whose word length is at most $|w|$. The role guard and current bit therefore imply the entire translated-window premise. Block soundness restores the tape and advances the head one cell. The new preceding suffix is $\mathrm{tail}_2(w)$, exactly the checked suffix of s. Its source node is the smaller child, so the induction hypothesis constructs the remaining chain. Prepending this block constructs the desired witness. Positivity holds for every internal block; adding the primitive transition makes the complete prefix positive even if the internal chain is empty. □

At the root the structural check fixes state 84 and suffix [1]. Thus the tape premise is $t(a-1) = 1$ plus the source-path bits beginning at head a; arbitrary exterior cells remain framed. Source-path binding must be included in the final theorem, not inferred from a role count. A path-driven version identifies exactly which leaf the prefix reaches.

Independent replay checks all 4,751 source paths and 170,627 literal transitions. The exact maximum is **56 steps through the first primitive transition**, at most 55 internal steps. A weighted role-DAG computation independently agrees. Separately, all 281 blocks pass and 1,124 malformed block cases fail. These finite checks support the induction; they do not compile its universal statement or prove full primitive semantics.

The 180-role quotient from the earlier follow-up remains optional and must retain unions of suffix guards. Changed path data on return, primitive execution, register boxes and complete site coverage still require adapters. Neither 281 block contracts nor the 56-step prefix bound replaces the 6,176 original runtime obligations. The earlier q2 return scanner is complementary, not superseded.

Concrete credit is now available as an input. Both new credit files use actual Cantor coding, with DROP tag 0 and NIP tag 1. Lowering a selected NIP to DROP strictly lowers its candidate index; retaining its position makes this occurrence map injective. The inverse restores that position. Prefix closure of the literal schedule $0, 1, 2, \ldots$ places each image earlier, proving nonnegative credit at each prefix and positive available credit immediately before any NIP. The checked packet records compiler and recursive-check success with five closed assumption reports.

The missing simplification is a source-scheduler adapter identifying actual refill, decoded command order and command occurrences with that concrete prefix model. It must preserve the opaque tail, top/cache and physical frame. Skipping predecessors or discarding occurrence positions would invalidate the counting argument. These syntax/counting proofs alone establish neither physical command correctness nor initialization, binary simulation or logical calibration of ZC215.

6 Integration priorities and boundaries worth retaining

Reuse existing marker proofs and the Core bridge. The shared native checker verifies incoming marks, noninitial distinct roles and exact finite row conditions. Its preceding-bit invariant holds initially on every tape; one- or three-step whole-tape-restoring blocks give $n \leq m \leq 3n$. Totalized semantics require absorbing halt and $\phi(q) = \text{halt} \Leftrightarrow q = \text{halt}$ for reflection. Closure checks find 215 complete working states and no nonhalt hole.

M124 and FM215 already have native Lean all-tape theorems; the generic `CoreNativeBridge.blank_iff` already supplies table/renaming transport. Their remaining work is semantic-interface matching, exact finite table compatibility and endpoint replay, not automatically a new native proof or general zipper bridge. Match state types, action representations, tape centering where needed, and designated/partial/missing-row halting. A matching M125 Lean instance is not yet established here. The new Coq checker is not a cross-kernel Lean proof. CM215 retains the shortest demonstrated exact dense-machine integration, not a completed full endpoint replay.

Keep transport APIs small and exact. Context215 reuses positive whole-tape stuttering with old-history coverage. Fully replayed blocks may contain changed rows if exact endpoints agree and no interior halt occurs. This relaxation is uncompiled and has no measured saving for the six blocks.

Logical transport fixes carrier, implication, truth and binders. Derivation induction uses target axiom admissibility and preserves truth, MP and theorem-only GEN; mutual admissibility gives equal theorem predicates. The uncompiled C222 adapter must retain every constructor, gate and dominance condition without borrowing removed axioms.

Do not count old work twice. The earlier scanner, DAG well-formedness fold, named calibration and terminal-drain proposals remain pending. Sparse-seed and padding ideas overlap that earlier work. Raw upper soundness must still cover arbitrary numerical codes and aliases; removing irreflexivity from one helper does not remove it from code-1 falsity or the full seed argument.

Ranked next steps.

1. **Integrate the checked normalized source theorem.** Preserve public interfaces, generators and machine bytes; replay under the maintained compiler and rebuild `Correct215` and `CorrectDense215`, with final assumption reports. Keep Lean and Rocq adoption claims separate.
2. **Prove lookup coverage as the generic source API.** Compile the positive-trie concrete theorem, then compare list, balanced and trie layouts under one clean/incremental protocol. Include construction and elaboration costs. Only then adapt the normalized lane.
3. **Generate the decoder Chain from structural acceptance.** Use child-node descent and exact suffix guards above, then connect primitive and runtime interfaces. Reuse the checked finite-window library rather than reproving it.
4. **Treat the 189-core as a research option.** Today's identical graphs favor direct equality and the compiled 1,426-entry route. Macros may help future non-identical loops, but require source-graph and macro refinement; fewer vertices implies no saving in total obligations, code or build cost.
5. **Adapt the concrete credit proof to the scheduler.** Use occurrence-level identity and actual refill order; do not repeat an abstract injectivity proof whose concrete premise is already discharged.

The bounded searches report 133,827,040 replacement-action tests across 21 tables and 457,682,472 assignments across 18 inputs, with no accepted fold. Neither was rerun here. Their spaces differ; counts cannot be added as distinct complete machines. They prove no 214-state lower bound; change the grammar or invariant precision before repeating them.

7 Pinned provenance and reproducibility

All links below refer to the fixed review commit `aa66d0a33f1f8237a348899eb8095b045a502832`, whose sole parent is `0359ebda60aaf52758358e1450a04988136db67e`. Arrival in `docs/incoming` is not production adoption. The three packages were prepared using older source pins; current principal inputs were separately rebound at this cutoff.

Affine proof cores and transport. Exact archive at the review commit. Git blob: `17321c28fc63f3dcdc895fa468eb3cad85d6cabe`. SHA-256: `4b9bf197546985b2c298b15d33bcf256e892a76a749ed1b367fe96aa4c536d46`. All 104 manifest payloads match. Sources include the canonical/affine graph data and `AxiomTransport.lean`. Current `L2Lib222.lean` has Git blob `e37f9ec83666edb39579240b2a28ad150aaadbf8`; see its existing `P_step` and `pair162` before adding new arithmetic infrastructure.

Certificate refinements. Exact archive at the review commit. Git blob: `7e969e67dfbd88a8b6aca044ce1541115de2f897`. SHA-256: `ae5edcd5fa0496d3fa13178a27b38108ad33a2301d970ebb5f22b968b1f6db0e`. All 105 manifest payloads match. Relevant modules are `BalancedBisim.v`, `Source215Balanced.v`, `MarkerFold.v`, `MarkerInstances.v` and `CantorCredit.v`. Fourteen balanced source/log pairs bind; generic balanced, concrete marker and concrete credit recursive checks are recorded separately.

Checked simplifications. Exact archive at the review commit. Git blob: `5e489899fef825c91bcbf2889d68ece6238f2251`. SHA-256: `18ec63a7a07bb5614e5002261fe1ce02f219ae0925e7b713df67868663bdb60b`. All 193 manifest payloads match. Relevant modules are `NormalizedBisim.v`, `NormalizedCert215.v`, `NormalizedSource215.v`, `FramedWindow.lean`, `DecoderCertificate.lean`, `K215Decoder.lean` and `TagLowering.v`. Twelve normalized source hashes, thirteen primary command/log pairs, two facade/client pairs and three decoder source/log pairs bind to their receipts.

Important direct bindings. The shared normalized JSON hashes to `6d30d89253629525b0af3066c7fec67b052c07413938fc7a94dd96ab1ff406e2`. The native K215 table hashes to `9683d6ed9628e6ce407c315b08f5da3d063da94e92706f275ca0f33e8272e4b2`. Current `Prog215.v` and `DagPerm.v` Git blobs are respectively `c74df29d0e0d3c9c91d46b3cda73c3563b7f2058` and `74a1715ff35e2b1abbf0cb5f58efdf2aafb1ef6b`. The Coq 8.18 compatibility import in `RMEquiv.v` is disclosed and its compiled-source hash reconstructed, not a Rocq 9.1 build. The existing generic Core bridge has Git blob `88c8a25edcfd8dab49ebdde6fda01349df35b358`; the candidate-bridge addendum supplies exact M124/FM215 pins and interface differences.

Local evidence. Owned audits rebuilt both canonical graphs, checked all 1,059 drain summaries, matrices and literal paths, and tested all 178 new loop occurrences on 26,166 register assignments, replaying 3,138,765 original checkpoint transitions and comparing all nine outputs. The compact 60,401-byte core independently expands through 33 shared matrices to the same full circuit; this is serialization size only. A separate verifier repeats the literal and matrix checks, adding 13,350 regressions, 2,157,631 checkpoint transitions and 89,478 affine steps. It verifies initial identity, literal halt and full coverage without using rank witnesses. The two regression sets are not claimed disjoint. Trie checks establish exact reindexing and reject corrupted inputs. Decoder checks bind every literal action, role, source path and block. All these are finite corroboration; the universal ordinary proofs are stated separately above.

Reproduction and limits. The accompanying review contains inert source/data copies, owned audit scripts, source pins and result manifests. Run the owned graph, trie and decoder audits, not received programs. Build this document with `build.sh`; the optional `K215_TEXFORMATS` variable selects an existing TeX format tree. Rendered-page QA and deliverable hashes accompany the report. Compiled objects were not reproduced, proof assistants were not invoked, and no full endpoint or timing baseline was rerun. These boundaries are part of the result, not deferred claims of success.